

CSE233 Week 12: Operator Overloading

This week, we will continue discussing classes and move to discussing operator overloading. In C++, we can overload operators (like +, -, =) to allow them to be used on objects. We'll also discuss friend functions, which may be needed for implementing the I/O operators (<< and >>). Note: I'll be using the term "free function" to refer to functions that are not class member functions.

Function Overloading

Recall the C++ supports function overloading. Function overloading allows the same function name to be used for different sets of parameters. The function overloads are differentiated by the types and number of parameters. The return type has no impact on whether the overload is legal. Here is a basic example of function overloading:

```
// 1.
int add(int x, int y) { return x + y; }
// 2.
double add(int x, double y) { return x + y; }
// 3.
double add(double x, int y) { return x + y; }
// 4.
double add(double x, double y) { return x + y; }

int main() {
    // calls 1
    int a = add(1, 1);
    // calls 2
    double b = add(1, 1.0);
    // calls 3
    double c = add(1.0, 1);
    // calls 4
    double d = add(1.0, 1.0);

    return 0;
}
```

Operator Overloading

Operator overloading allows operators, like +, -, <, >, ==, etc. to be used for objects of a class (or struct). Let's look at some examples. You may have noticed that the `double` data type does not always have perfect decimals. This has to do with how decimal values are stored at the hardware level. This is not sufficient for storing monetary values. It is unacceptable in finance applications to have imprecise values. So, we will make a `Money` class to store US dollars and cents using integers to ensure the values are correct. However, we want to still be able to use the `Money` type like an integer or double where we can add, subtract, compare, etc. the variables. To achieve this, we must overload the operators. We aren't going to consider negative values in the example, since it would add a bunch of extra calculations and checks. This would distract from implementing the operators, which is the focus of this week.

One thing to remember with overloaded operators is that the left-hand side of an expression takes the place of the object being used to call the operator. For example, consider:

```
c = a + b;
```

If we defined a member function `add()` as well, this would be equivalent to:

```
c = a.add(b)
```

Now, let's consider the `Money` class:

```
class Money {
public:
    // constructor, default values of $0.00
    Money(unsigned long d = 0, unsigned short c = 0);

    // comparison operators, should always be const
    bool operator==(const Money& rhs) const;
    bool operator<(const Money& rhs) const;

    // assignment operators
    Money& operator+=(const Money& rhs);
    Money& operator-=(const Money& rhs);
```

```

// arithmetic operators
Money operator+(const Money& rhs) const;
Money operator-(const Money& rhs) const;

// other members
void addDollars(unsigned long d);
void addCents(unsigned long c);
void print() const;
std::string toString() const;
private:
    // the dollar amount, we use a very large data type
    unsigned long dollars;
    // cents only range from 0 to 100, so we can use the
    // smallest integer type, short
    unsigned short cents;
};
// techically, these could be members as well, but they
// can be implemented as free functions using == and <
// free functions typically have better performance than
// member functions
bool operator!=(const Money& lhs, const Money& rhs) {
    return !(lhs == rhs);
}

bool operator<=(const Money& lhs, const Money& rhs) {
    return (lhs < rhs) || (lhs == rhs);
}

bool operator>(const Money& lhs, const Money& rhs) {
    return !(lhs < rhs) && !(lhs == rhs);
}

bool operator>=(const Money& lhs, const Money& rhs) {
    return !(lhs < rhs);
}

// implement constructor
Money::Money(unsigned long d, unsigned short c) {
    dollars = d;
    // handle case where cents is greater than 100
    // if the cents is less than 100, these won't make a difference
    dollars += c / 100;
    cents = c % 100;
}

// implement comparison operators
bool Money::operator==(const Money& rhs) const {
    // if all the fields are the same, the money objects are equal
    return (dollars == rhs.dollars) &&
        (cents == rhs.cents);
}

```

```

}

bool Money::operator<(const Money& rhs) const {
    // if the dollars are less than rhs's dollars
    // this is less than the rhs
    if (dollars < rhs.dollars) return true;
    // if both dollar amounts are equal, we need to compare cents
    else if (dollars == rhs.dollars) {
        if (cents < rhs.cents) return true;
        else return false;
    }
    // otherwise, rhs.dollars are less and rhs is less overall
    else return false;
}

// implement the assignment operators
Money& Money::operator+=(const Money& rhs) {
    // add the rhs cents
    cents += rhs.cents;
    // add the rhs dollars
    dollars += rhs.dollars;
    // if the cents are now over 100, this will handle the rollover
    dollars += cents / 100;
    cents %= 100;

    return *this;
}

Money& Money::operator-=(const Money& rhs) {
    // rather than actually handling negatives, just force
    // the amount to 0
    if (dollars < rhs.dollars) {
        dollars = 0;
        cents = 0;
    }
    else if (dollars == rhs.dollars) {
        if (cents < rhs.cents) {
            dollars = 0;
            cents = 0;
        }
        else {
            dollars = 0;
            cents -= rhs.cents;
        }
    }
    else {
        dollars -= rhs.dollars;
        if (rhs.cents < cents) {
            cents -= rhs.cents;
        }
        else {

```

```

        if (cents == 0) {
            cents = 100 - rhs.cents;
        }
        else {
            cents = rhs.cents - cents;
        }
        --dollars;
    }
}

return *this;
}

// implement the arithmetic operators
Money Money::operator+(const Money& rhs) const {
    Money returnValue(dollars, cents);
    returnValue += rhs;
    return returnValue;
}

Money Money::operator-(const Money& rhs) const {
    Money returnValue(dollars, cents);
    returnValue -= rhs;
    return returnValue;
}

// other members
void Money::addDollars(unsigned long d) {
    dollars += d;
}

void Money::addCents(unsigned long c) {
    cents += c;
    dollars += c / 100;
    cents %= 100;
}

void Money::print() const {
    std::cout << '$' << dollars << '.'
    << cents / 10 << cents % 10;
}

std::string Money::toString() const {
    std::string str = "$";
    std::string dollarStr = std::to_string(dollars);
    char centStr[3];
    centStr[0] = cents / 10;
    centStr[1] = cents % 10;
    str += dollarStr;
    str += ".";

```

```
    str += centStr;  
    return str;  
}
```

Let's look at the typical structure of the signatures for the various operator overloads. Boolean operators (`==`, `<`) have the form of:

```
bool operator==(const DataType& rhs) const;
```

Comparison operators should not modify the data of an object, so they are `const`. The object being compared to should be passed as a `const` reference since it also will not be modified.

Next, the assignment operators look like this:

```
DataType& operator+=(const DataType& rhs);
```

These modify the calling object, so they are not `const`. The object being added, subtracted, etc. from the calling object is not modified, so it should be a constant reference. These operators return a reference to the calling object. The `return *this;` at the end of the function is what does so.

The arithmetic operators look like this:

```
DataType operator+(const DataType& rhs) const;
```

These do not modify the calling object. Instead, they return a new object that is the result of the calling object and object passed as the parameter being combined.

There is another type of operator that cannot be demonstrated using the money example: the subscript operators. These are the operators that are used to access an array's elements. If the data type being defined by the class is array-like, these can be used. Let's consider this example of an `IntArray` class:

```
const int MAX_ARRAY_SIZE = 256;  
  
class IntArray {  
public:  
    IntArray();  
    int maxSize() const;
```

```

    int size() const;
    bool empty() const;
    bool full() const;
    // append returns true if there was room to add a new value
    // or false otherwise
    bool append(int value);
    // overloaded subscript operators
    // this one is const (read only)
    int operator[](int i) const;
    // this one returns a reference,
    // so it can be used to update the value in the array
    int& operator[](int i);
    void printArray() const;
private:
    int data[MAX_ARRAY_SIZE];
    int currentSize;
};
// other members omitted for brevity

// readonly subscript operator
int IntArray::operator[](int i) const {
    // if the index is greater than current size,
    // the value at the index is meaningless
    if (i < currentSize) {
        return data[i];
    }
    else {
        std::cout << "Invalid index\n";
        exit(1);
    }
}

// read/write subscript operator
int& IntArray::operator[](int i) {
    // since this is used to write and read data
    // we can't guard it as strictly as the read only one
    if (i < MAX_ARRAY_SIZE) {
        // if the index is greater than the current size
        // update the current size to be the index
        if (currentSize < i) {
            currentSize = i;
        }
        return data[i];
    }
    else {
        std::cout << "Invalid index\n";
        exit(1);
    }
}
}

```

There are two versions of the subscript operator. One is `const`, and can only be used for reading a value, i.e., `int x = array[0]`. The other one is used for reading or writing, i.e., `array[0] = 10`, so it has to return a reference so the value can be written.

```
int main() {
    IntArray array;
    for (int i = 0; i < 10; ++i) {
        array.append(i);
    }
    for (int i = 0; i < array.size(); ++i) {
        // using operator[] to print the values
        std::cout << array[i] << "\n";
    }
    // using operator[] to modify a value
    array[10] = 10;

    return 0;
}
```

The only operators left to discuss are the insertion and extraction operators `<<` and `>>`, but first we need to discuss friend functions.

Friend Functions

Friend functions are functions that are not members of the class (i.e., not called with the `object.function()` syntax), but have access to an object's private members. Friend functions must have their declaration placed in the body of the class declaration, and have the `friend` keyword before them:

```
const int MAX_ARRAY_SIZE = 256;
class IntArray {
public:
    IntArray();
    int maxSize() const;
    int size() const;
    int operator[](int i) const;
    int& operator[](int i);
    // member function, no parameter
    void printArray() const;
    // friend function, has a parameter
    friend void printArray(const IntArray &array);
private:
    int data[MAX_ARRAY_SIZE];
    int currentSize;
```

```

};

void IntArray::printArray() const {
    bool printComma;
    // normal member function, access member variables directly
    for (int i = 0; i < currentSize; ++i) {
        if (printComma) {
            std::cout << ", ";
        }
        std::cout << data[i];
        printComma = true;
    }
}

// friend function does not have IntArray:: or friend specifier when
// being defined
void printArray(const IntArray &array) {
    bool printComma;
    // friends do not have direct access to class member variables
    // they can access the member variables of its parameters,
    // even if they are private, though
    for (int i = 0; i < array.currentSize; ++i) {
        if (printComma) {
            std::cout << ", ";
        }
        std::cout << array.data[i];
        printComma = true;
    }
}

```

In general, friend functions are best avoided whenever possible. Any time a friend has to be used, it is typically indicative of a design flaw. However, sometimes they are needed, or far too convenient to not be used.

Overloading the Insertion and Extraction Operators

The insertion `<<` and extraction `>>` operators can be implemented as free functions or friend functions. They cannot be implemented as member functions. If the operator does not need to access the private members of the class, it is better to implement it as a free function. Sometimes, it is not possible to do so, so in this case a friend function must be used.

The overloaded insertion operator has a signature like this:

```
std::ostream& operator<<(std::ostream& out, const DataType& rhs);
```

The parameter `out` is passed by reference returned by the function. The extraction operator is defined similarly:

```
std::istream& operator<<(std::istream& out, DataType& rhs);
```

The primary difference is that the second parameter is no longer `const`, since the extraction operator (i.e., input operator) needs to modify it. And, of course, the return type and first parameter are `std::istream&` instead of `std::ostream&`. These are the base types for `std::cin` and `std::cout`, as well as file input and output streams. This allows the same operator overload to be used for reading and writing to the console and files. Now, let's look at a concrete example with the `IntArray` class. The `IntArray` class has a good enough interface to not need the operators to be friend functions, but I defined them as friends just to demonstrate the concept again.

```
const int MAX_ARRAY_SIZE = 256;

class IntArray {
public:
    IntArray();
    int maxSize() const;
    int size() const;
    bool empty() const;
    bool full() const;
    // append returns true if there was room to add a new value
    // or false otherwise
    bool append(int value);
    // overloaded subscript operators
    // this one is const (read only)
    int operator[](int i) const;
    // this one returns a reference,
    // so it can be used to update the value in the array
    int& operator[](int i);
    // member function, no parameter
    void printArray() const;
    // friend function, has a parameter
    friend void printArray(const IntArray &array);
    // overloaded insertion operator
    friend std::ostream& operator<<(std::ostream& out, const IntArray&
    // overloaded extraction operator
    friend std::istream& operator>>(std::istream& in, IntArray& array)
```

```

private:
    int data[MAX_ARRAY_SIZE];
    int currentSize;
};

std::ostream& operator<<(std::ostream& out, const IntArray& array) {
    bool printComma;

    for (int i = 0; i < array.currentSize; ++i) {
        if (printComma) {
            std::cout << ", ";
        }
        out << array.data[i];
        printComma = true;
    }

    // overloaded << and >> always return the stream
    // that is passed as a parameter
    return out;
}

std::istream& operator>>(std::istream& in, IntArray& array) {
    std::string line;
    std::getline(in, line);
    // remove any leading spaces
    while (line[0] == ' ') {
        line.erase(0, 1);
    }
    int start = 0;
    int end = line.find(' ');
    while (end != std::string::npos) {
        // extract a number
        std::string number = line.substr(start, end - start);
        // convert it to an int and add it to the array
        array.data[array.currentSize] = std::stoi(number);
        ++(array.currentSize);
        // erase the number
        line.erase(start, end);
        // erase any extra spaces between numbers
        while (line[0] == ' ') {
            line.erase(0, 1);
        }
        // find the end of the next number
        end = line.find(' ');
    }

    return in;
}

```

Now, we can use the overloaded operators for our `IntArray` class.

```
int main() {
    IntArray array;

    // use overloaded operator >>
    std::cout << "Enter integers (separated by spaces).\n";
    std::cin >> array;

    // print array using overloaded operator <<
    std::cout << "You entered: ";
    std::cout << array << "\n";

    return 0;
}
```

Conclusion

Overloading operators allows us to create classes that are more natural to use. Think back to how C strings required functions like `strcat`, `strcpy`, and `strcmp` to do anything. Meanwhile, `std::string` implements `+`, `=` and `==` to make these operations more convenient. We looked at the common patterns for comparison, assignment, arithmetic, and I/O operators. We also discussed friend functions, which are free functions that have access to a class's private members.